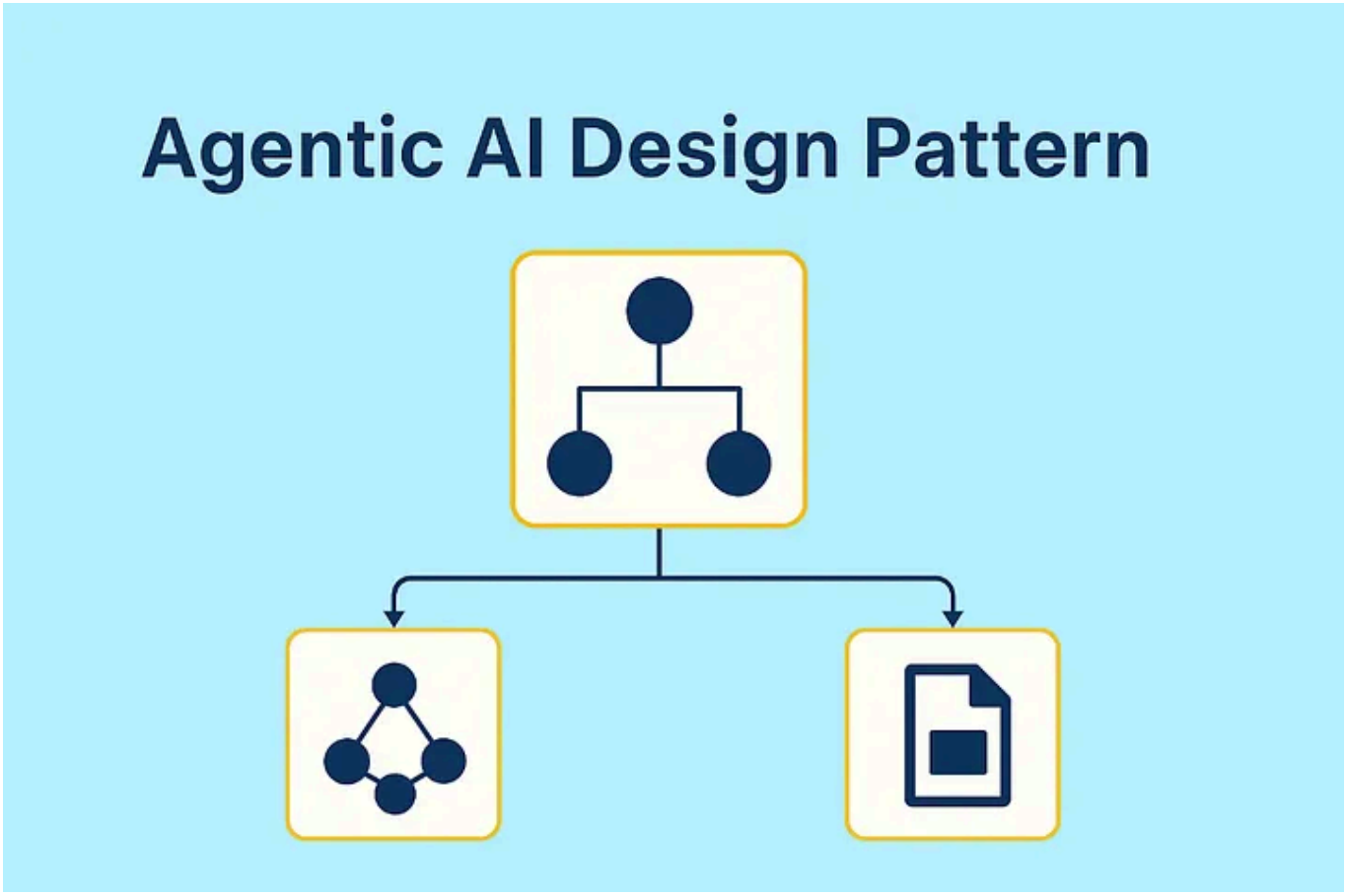
Agentic AI Design Patterns: Choosing the Right Multimodal & Multi-Agent Architecture (2022–2025)

This guide provides a comprehensive overview of Agentic AI design patterns that have emerged and matured from 2022 to 2025. Each pattern is presented with its architecture, performance characteristics, and specific use cases to help you select the appropriate pattern for your implementation.

Balaram Panda Follow 9 min read · Aug 15, 2025



Pattern Categories

1. **Single-Agent Reasoning Patterns:** CoT, ToT, GoT, Reflexion
2. **Multi-Agent Collaboration Patterns:** MAD, Collaborative Frameworks, Society of Minds
3. **Augmentation Patterns:** Tool-Use, Memory Systems
4. **Advanced Patterns:** MCTS-Based, Self-Improving Agents
5. **Hybrid Pattern:** Adaptive Reasoning Orchestrator (Proposed)

```
## Implementation

This article is based on our open-source library that standardizes these pattern
```

🚀 **GitHub**: https://github.com/balrm/agentic-patterns
📦 **Install**: `pip install agentic-patterns`

TLDR : Design Pattern Selection Guide

Agentic AI Pattern	Best For	Complexity	Cost	Speed	Accuracy
Chain-of-Thought	Math, Logic	Low	$	⚡ Fast	Med-High
Tree of Thoughts	Puzzles, Planning	Medium	$$$	🐢 Slow	High
Graph of Thoughts	Complex Problems	High	$$	🐢 Slow	Very High
Reflexion	Programming, QA	Medium	$$	⚡ Medium	Very High
Multi-Agent Debate	Complex Reasoning	Medium	$$$	🐢 Slow	Very High
Tool-Use	External Data, APIs	Low	$	⚡ +0.5-2s	Task-Specific
Memory Systems	Multi-session, Context	Medium	$$	⚡ Fast	High
MCTS-Based	SOTA Performance	High	$$$$	🐢 Very Slow	SOTA
Self-Improving	Long-term Systems	Very High	$$$	⚡ Medium	Improving
ARO (Proposed)	Production Systems	Variable	Optimal	⚡ Adaptive	Adaptive

Legend: Cost: $ = Low | $$ = Medium | $$$ = High | $$$$ = Very High
Speed: ⚡ = Fast Response | 🐢 = Slower Processing
Complexity Colors: Low Medium High Very High

Quick Selection Guide

- Budget-conscious → Chain-of-Thought or Tool-Use ($)
- Quality-critical → Reflexion or Multi-Agent Debate
- Time-sensitive → CoT, Tool-Use, or Memory Systems
- Research/SOTA → MCTS-Based or Self-Improving

Implementation: https://github.com/balrm/agentic-patterns

Design pattern selection guide by author

Single-Agent Reasoning Patterns

1. Chain-of-Thought (CoT)

Architecture: Linear reasoning chain where the model explicitly shows intermediate reasoning steps before reaching a conclusion.

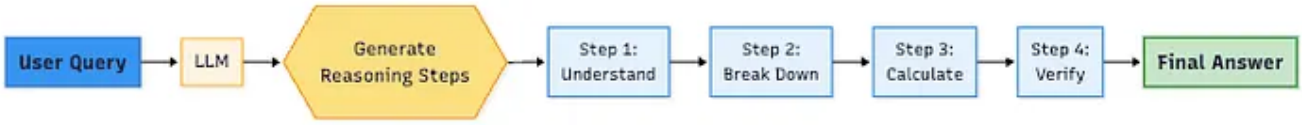


diagram by author

Variants:

- **Few-shot CoT:** Requires manual examples
- **Zero-shot CoT:** Uses prompts like "Let's think step by step"
- **Self-Consistency CoT:** Samples multiple reasoning paths and votes

Performance:

- MultiArith: 17.7% → 78.7% (Zero-shot CoT)[3] (https://arxiv.org/abs/2303.05398)
- GSM8K: Additional 17.9% improvement with Self-Consistency[4]
- Requires 100B+ parameter models for optimal performance[2]

When to Use:

- Mathematical reasoning problems
- Multi-step logical deduction

- Tasks requiring interpretable reasoning
- When computational budget is limited (single inference)

Limitations:

- Single reasoning path (except Self-Consistency variant)
- Performance heavily dependent on model size
- Cannot backtrack from errors

2. Tree of Thoughts (ToT)

Architecture: Tree structure allowing exploration of multiple reasoning paths with explicit backtracking capability. Maintains a search tree where each node represents a partial solution.

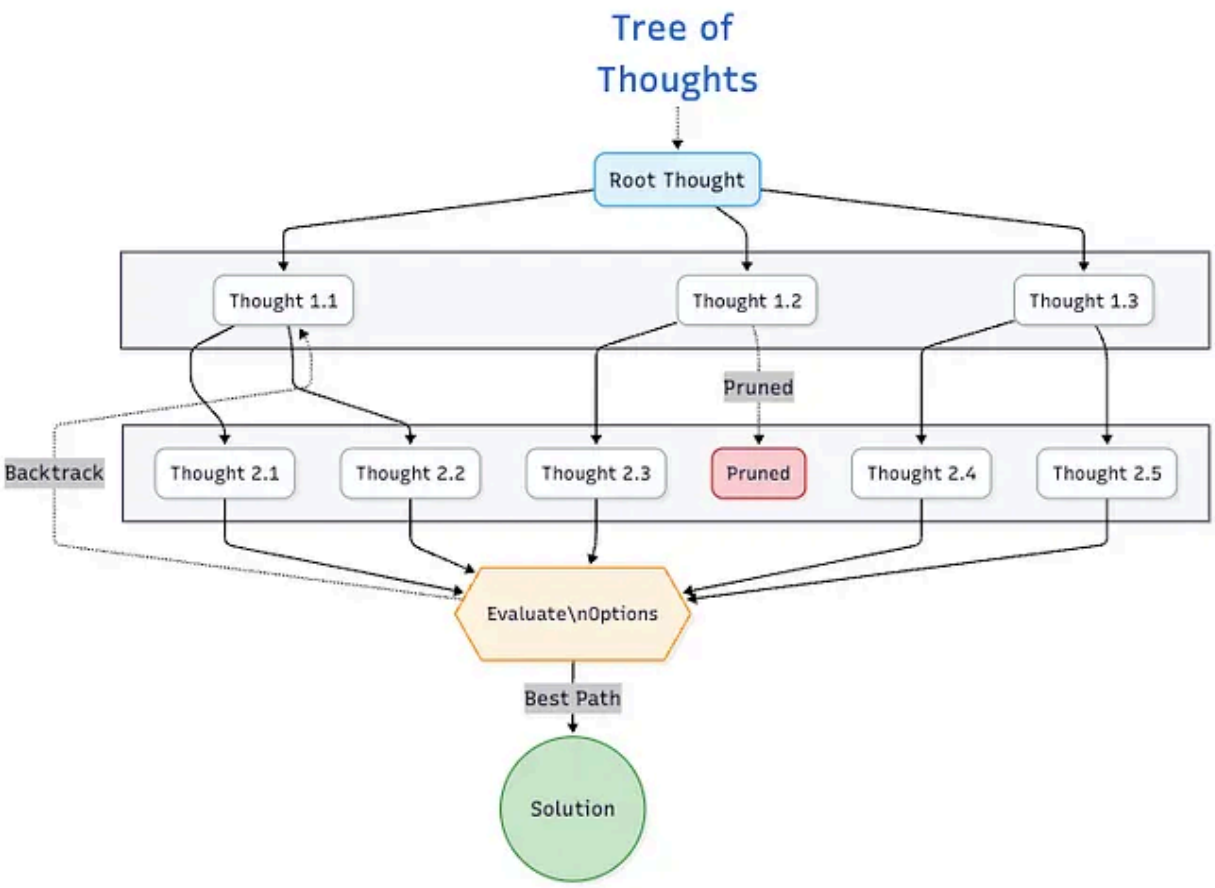


Image by author

Performance:

- Game of 24: 74% success rate (vs 4% with CoT)[5]
- Creative writing: 5.3x improvement in coherence
- Cost: b×d LLM calls (b=branching factor, d=depth)

When to Use:

- Complex puzzles and constraint satisfaction problems
- Creative tasks requiring exploration (story writing, poetry)
- Problems where initial approaches might fail

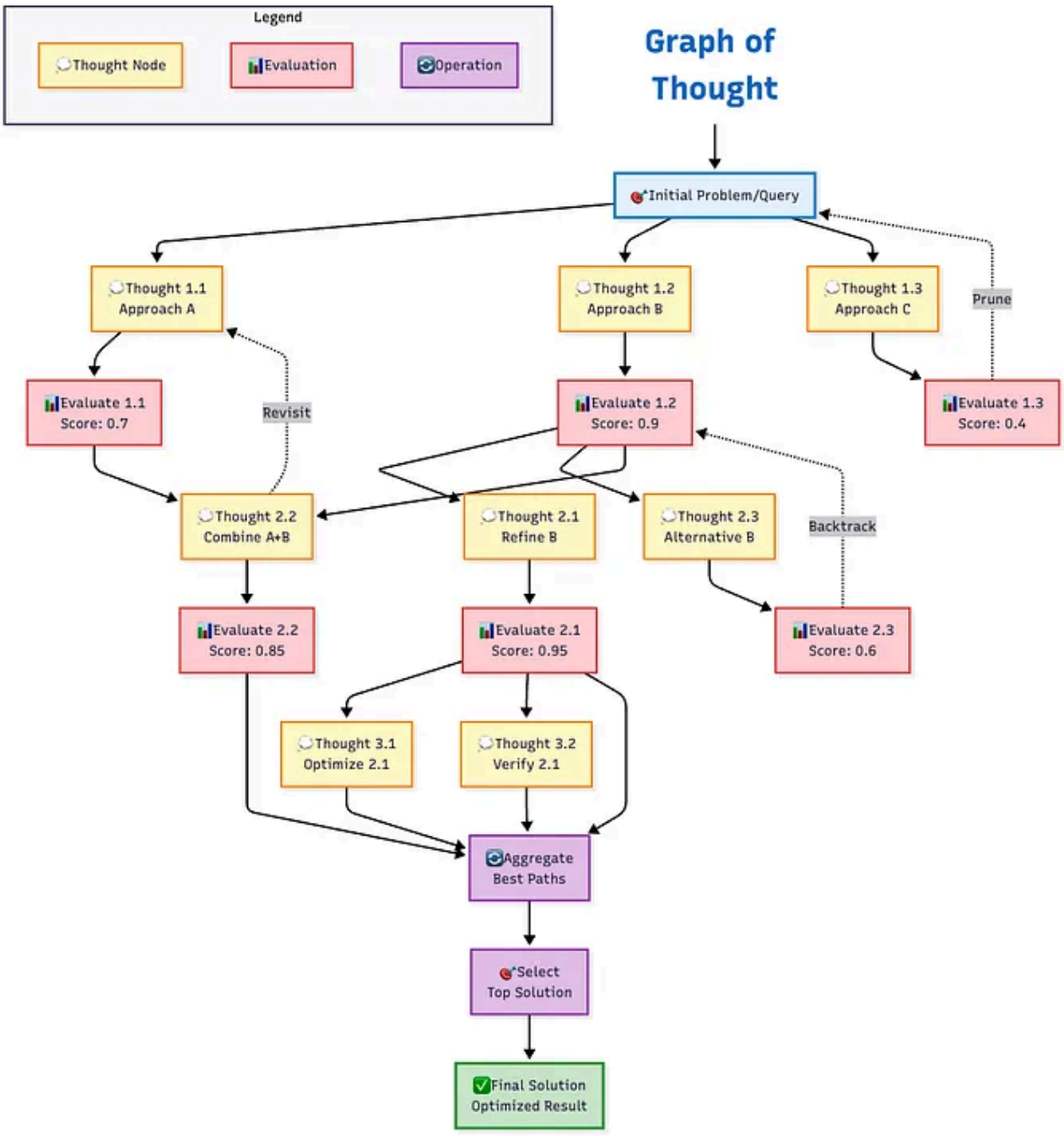
- When you can afford multiple LLM calls for better accuracy

Limitations:

- High computational cost (5–10x more than CoT)
- Requires explicit evaluation function
- Not suitable for real-time applications

3. Graph of Thoughts (GoT)

Architecture: Generalizes ToT by allowing arbitrary graph structures. Supports operations like thought merging, refining, and splitting. Enables reuse of intermediate thoughts.



Diagrams by author

Performance:

- 62% quality improvement over ToT[6]
- 31% cost reduction compared to ToT
- Better for problems with shared sub-problems

When to Use:

- Problems with overlapping subproblems
- Tasks benefiting from thought aggregation
- Complex planning with multiple valid paths
- When thought reuse can reduce costs

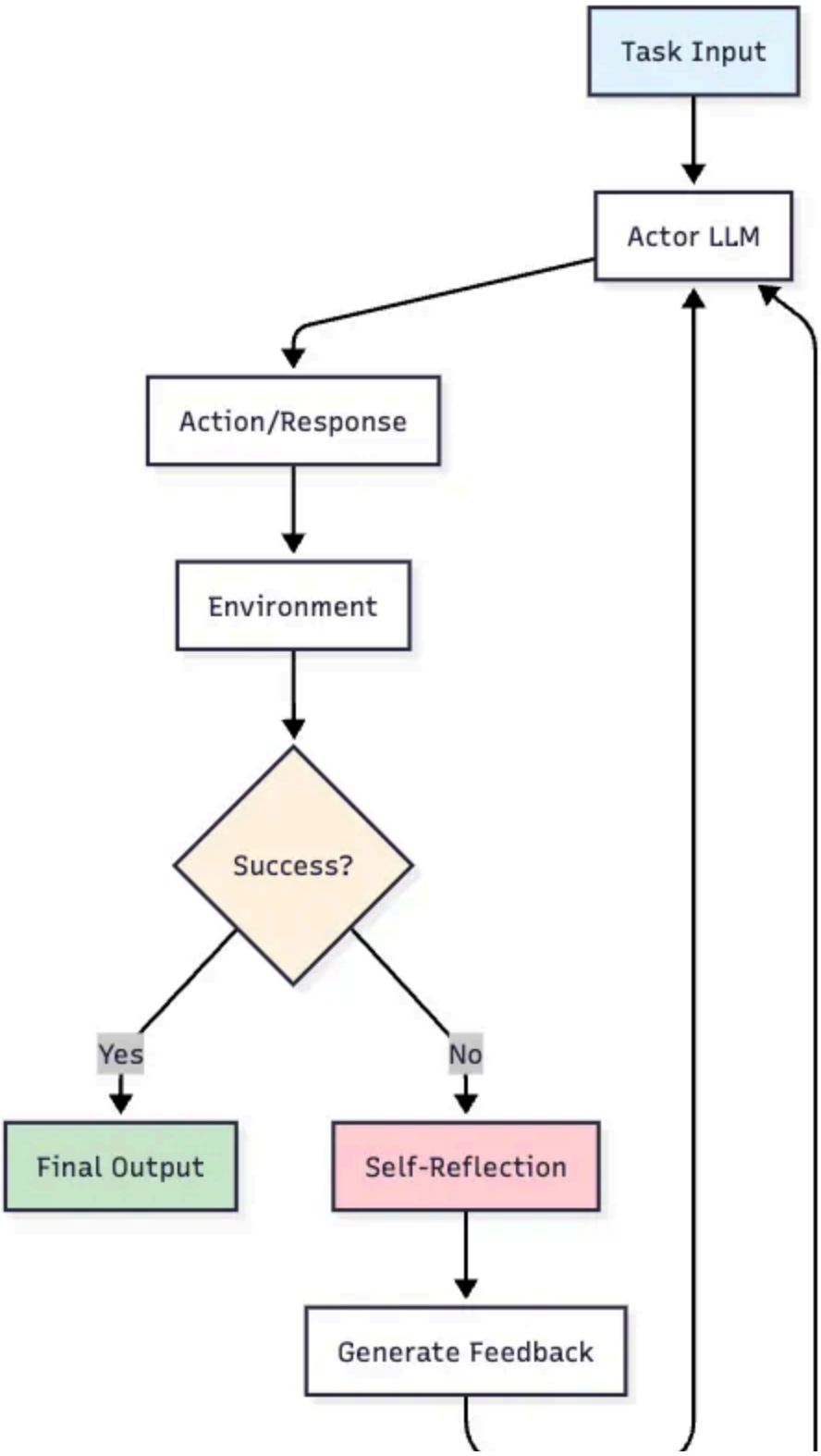
Limitations:

- Complex implementation
- Requires sophisticated thought merging logic
- Higher overhead for simple problems

4. Reflexion

Architecture: Three-component system with Actor (generates actions), Evaluator (assesses outcomes), and Self-Reflection module (generates improvement feedback). Maintains episodic memory across trials.

Reflexion Pattern



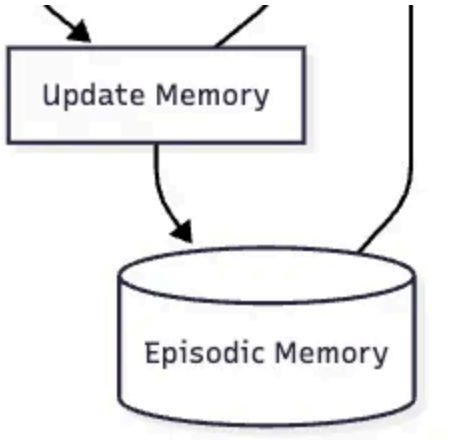


Image by author

Performance:

- HumanEval: 88% pass@1 (vs GPT-4 67%)[7],[8]
- AlfWorld: 130/134 tasks completed
- Typically converges in 3–5 iterations

When to Use:

- Programming tasks with test cases
- Sequential decision-making problems
- Tasks with clear success metrics
- When learning from failure is valuable
- Iterative improvement scenarios

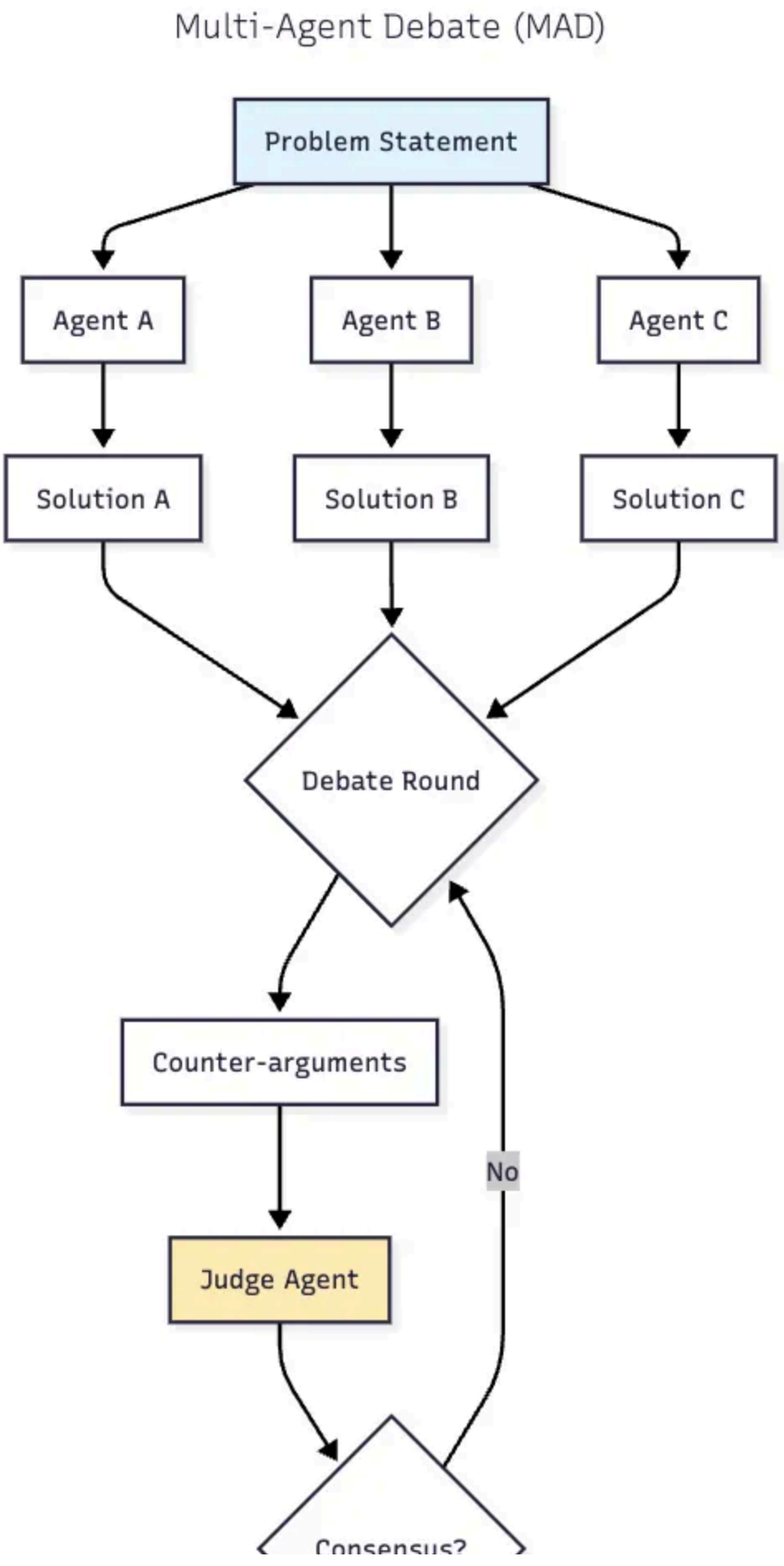
Limitations:

- Requires multiple trials
- Needs reliable self-evaluation

- Not suitable for one-shot tasks

Multi-Agent Collaboration Patterns

5. Multi-Agent Debate (MAD)


Multi-Agent Debate (MAD)
Problem Statement
Agent A
Agent B
Agent C
Solution A
Solution B
Solution C
Debate Round
Counter-arguments
No
Judge Agent
Consensus?

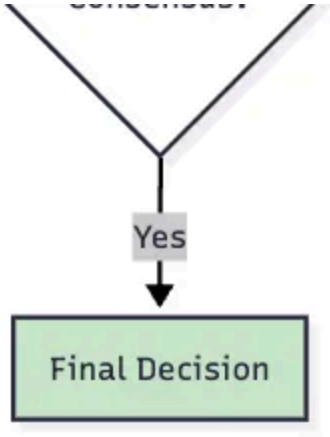


Image by author

Architecture: Multiple agents propose solutions, engage in structured debate rounds, with optional judge agent for final decision. Includes mechanisms for handling agreement intensity.

Performance:

- 15–47% improvement over single agent[9]
- Optimal configuration: 3–5 agents
- Cost: 5–15 LLM calls per question

When to Use:

- Complex reasoning requiring diverse perspectives
- Problems prone to single-agent bias

- Tasks benefiting from adversarial validation
- When accuracy is more important than cost

Limitations:

- 3–5x more expensive than single agent
- Requires careful hyperparameter tuning
- Can converge to wrong consensus without proper setup

Augmentation Patterns

6. Tool-Use Pattern (Function Calling)

Architecture: LLM decides when to call external tools (calculator, search, database, code execution). Modern implementations use function calling with structured outputs.

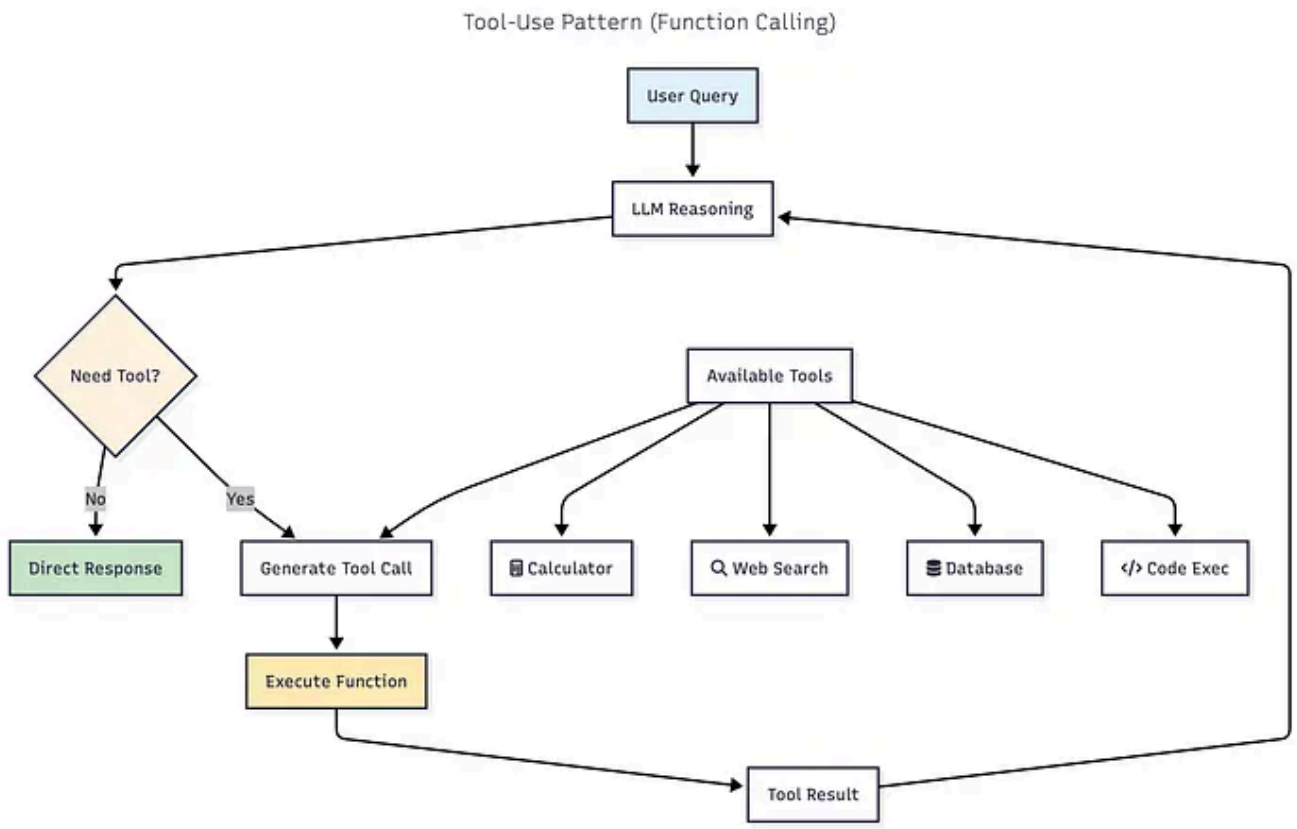


Image by author

Performance[13]**:**

- Math tasks: 25–40% improvement
- Real-time queries: 60–80% improvement
- Function calling accuracy: GPT-4o (84.1%), Claude-3.5 (82.9%)[14]
- Latency: +0.5–2.0 seconds per tool call

When to Use:

- Tasks requiring real-time data
- Mathematical computations
- Database queries

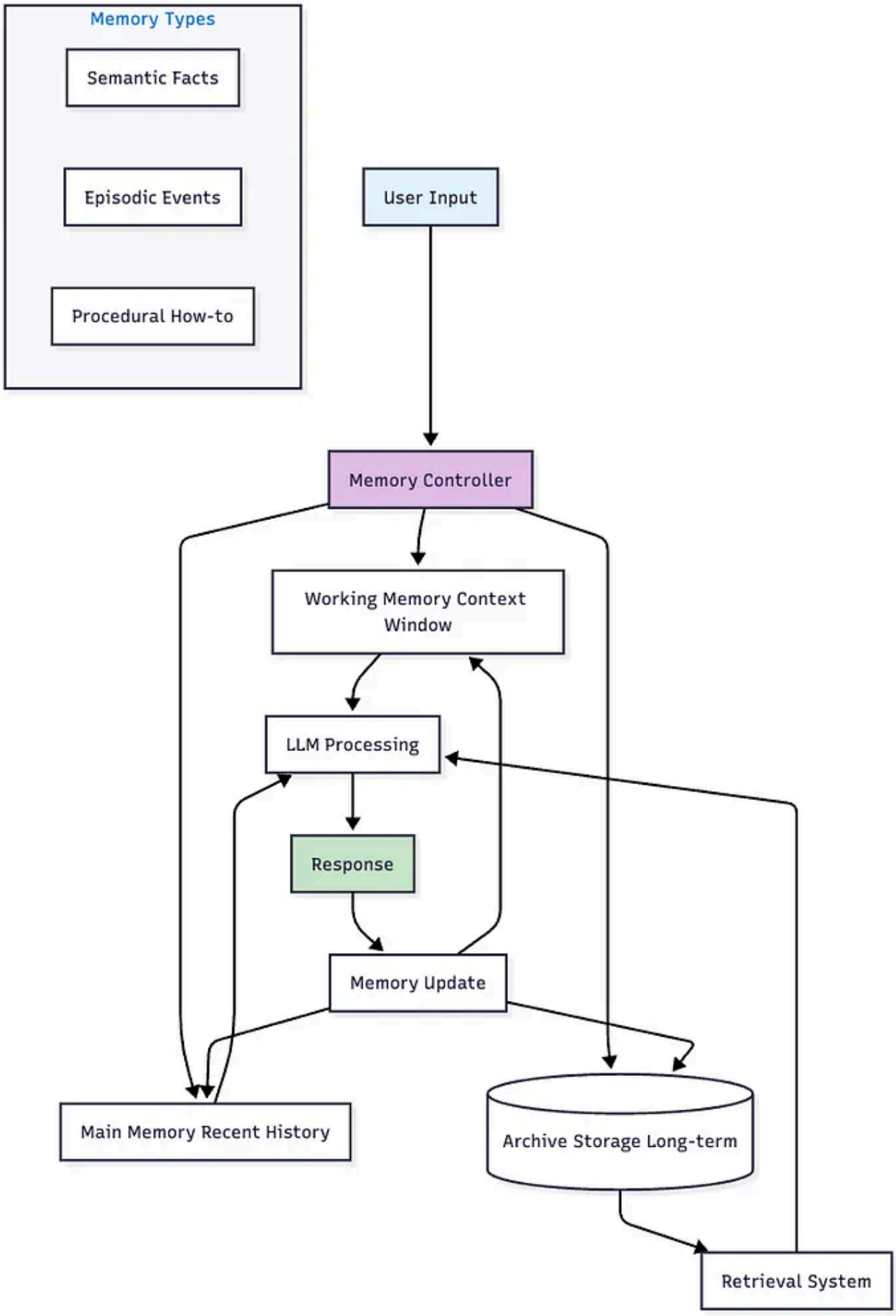
- Any task needing external capabilities
- When accuracy on specific domains is critical

Limitations:

- Added latency per tool call
- Potential for error propagation
- Requires careful tool design

7. Memory Systems

Architecture: Hierarchical memory inspired by OS design. Three tiers: Working Memory (active context), Main Memory (recent history), Archive (long-term storage with retrieval).


Memory Types
Semantic Facts
Episodic Events
Procedural How-to
User Input
Memory Controller
Working Memory Context Window
LLM Processing
Response
Memory Update
Main Memory Recent History
Archive Storage Long-term
Retrieval System

Key Implementations:

- **MemGPT:** Virtual context management, handles 100x context window[15]
- **Mem0:** 26% higher accuracy than OpenAI Memory, 91% lower p95 latency[16]

Performance by Memory Type[17]**:**

- Semantic memory: 40–60% improvement in factual consistency
- Episodic memory: 35–50% improvement in task completion
- Procedural memory: 20–35% improvement in complex execution

When to Use:

- Multi-session applications
- Tasks requiring long-term context
- Personalization systems
- Complex workflows with state
- Chat applications with history

Limitations:

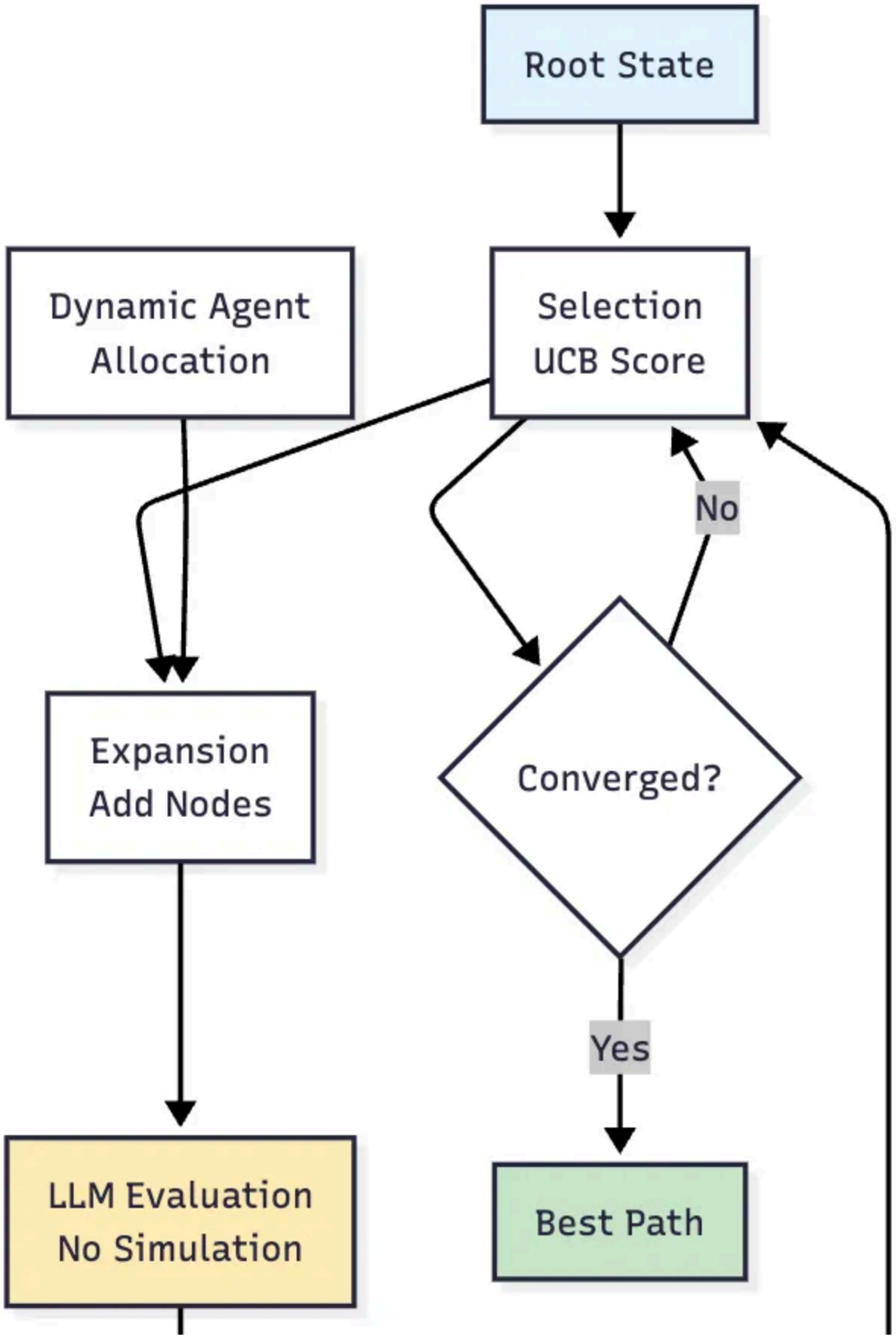
- Storage and retrieval overhead
- Complexity in memory management
- Potential for outdated information

Advanced Patterns

8. MCTS-Based Reasoning (MASTER)

Architecture: Monte Carlo Tree Search adapted for LLMs. Key innovation: eliminates simulation phase, using LLM self-evaluation instead. Dynamically adjusts agent count.

MCTS-Based Pattern (MASTER)



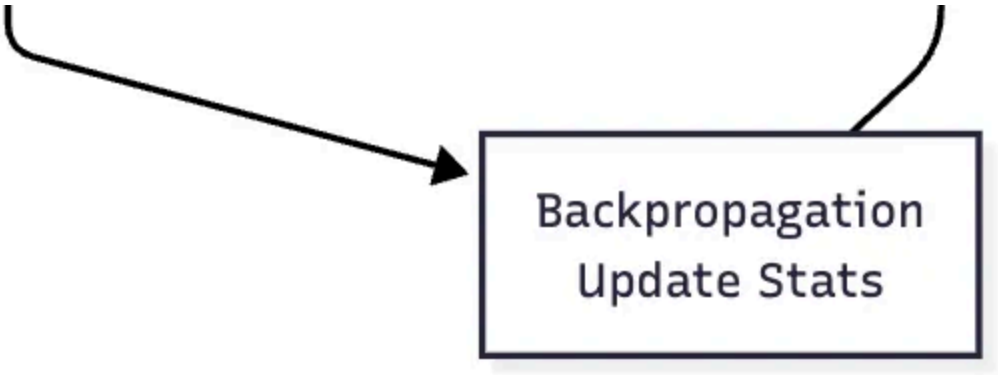


Image by author

Performance[18]**:**

- HotpotQA: 76% accuracy (SOTA)
- WebShop: 80% success rate (SOTA)
- 40.59% improvement over Zero-shot CoT[19]

When to Use:

- Complex multi-step reasoning
- Problems with large search spaces
- Tasks benefiting from lookahead
- When you need SOTA performance
- Research and competition scenarios

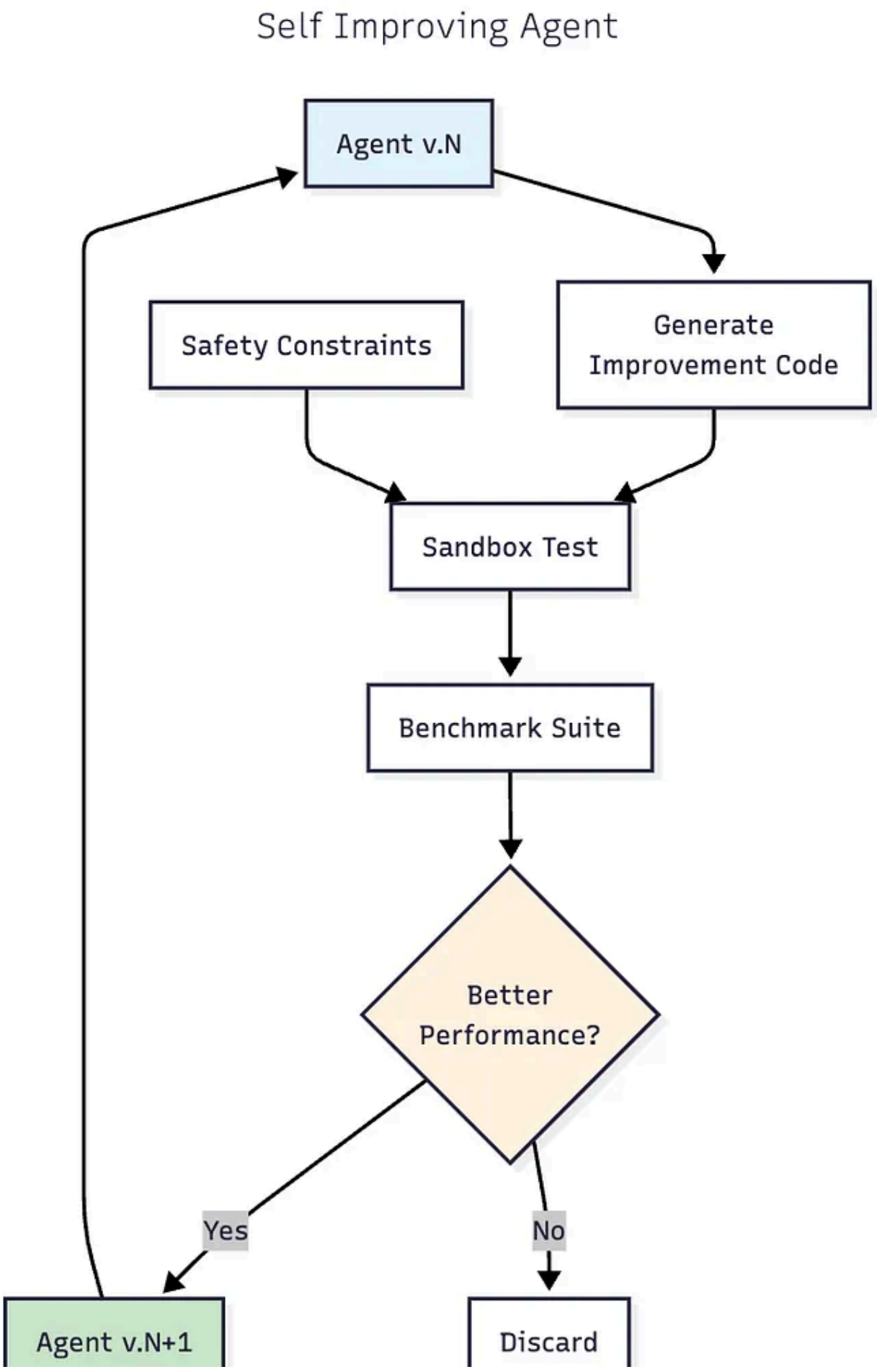
Limitations:

- High computational cost
- Complex implementation

- Requires good value estimation

9. Self-Improving Agents

Architecture: Agents that modify their own code/prompts based on performance. Includes sandboxed testing and empirical validation before deployment.


Self Improving Agent
Agent v.N
Safety Constraints
Generate Improvement Code
Sandbox Test
Benchmark Suite
Better Performance?
Yes
No
Agent v.N+1
Discard

Performance (Darwin Gödel Machine)[20]:

- SWE-bench: 20.0% → 50.0%
- Polyglot: 14.2% → 30.7%
- Continuous improvement over time

When to Use:

- Long-running systems
- Tasks with changing requirements
- Research environments
- When human supervision is limited
- Experimental AI systems

Limitations:

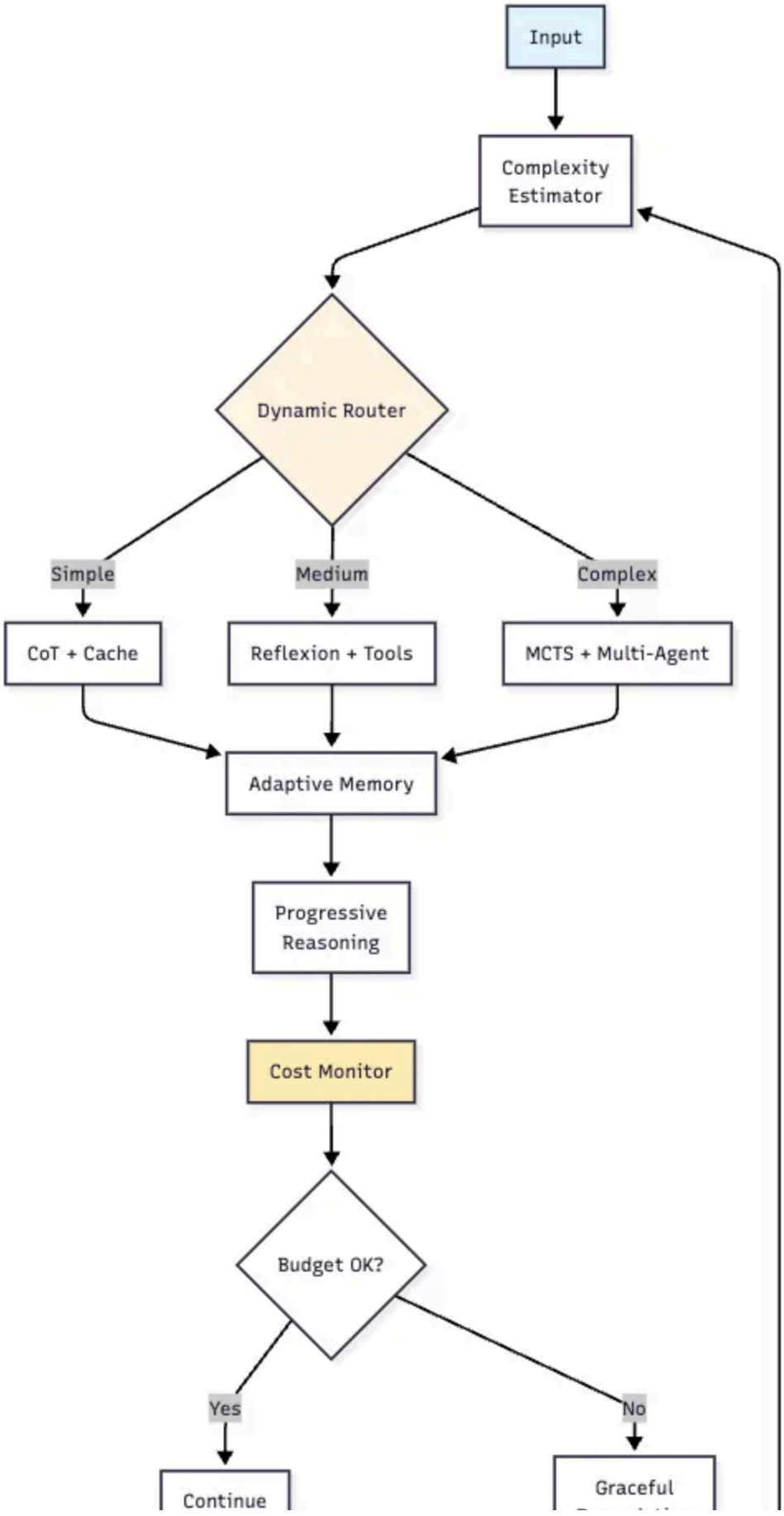
- Safety concerns
- Unpredictable behavior
- Requires extensive sandboxing
- Not suitable for production without safeguards

Hybrid Pattern

10. Adaptive Reasoning Orchestrator (ARO) — Proposed

Architecture: Meta-pattern that dynamically selects and combines other patterns based on task complexity, cost constraints, and performance requirements. Includes learning component for pattern selection improvement.

Adaptive Reasoning Orchestrator (ARO)-Proposed



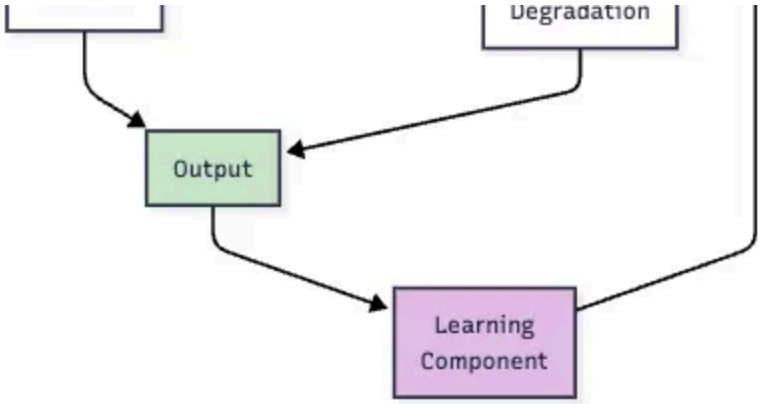


Image by author

Components:

- Complexity Estimator
- Dynamic Router
- Cost Monitor
- Performance Analytics
- Pattern Selection ML

Routing Logic:

- Simple tasks → CoT + Cache ($)
- Medium complexity → Reflexion + Tools ($)
- High complexity → MCTS + Multi-Agent ($$)

When to Use:

- Production systems with varied workloads
- When cost optimization is critical

- Applications with mixed complexity tasks
- Systems requiring adaptability
- Enterprise deployments

Advantages:

- Optimal cost/performance balance
- Learns from usage patterns
- Graceful degradation under load
- Unified interface for all patterns

Note: This is a proposed pattern based on analysis of existing patterns and production needs. Implementation would combine elements from established patterns with dynamic selection logic.

Decision Framework

Choose Single-Agent Patterns When:

- Task complexity is manageable by one model
- Interpretability is important
- Computational budget is limited
- Real-time response is needed

Choose Multi-Agent Patterns When:

- Problem benefits from diverse perspectives

- Accuracy is critical
- Complex domain expertise is required
- Cost is less important than performance

Choose Augmentation Patterns When:

- External capabilities are needed (tools)
- Long-term context is important (memory)
- Task-specific accuracy is critical

Choose Advanced Patterns When:

- SOTA performance is required
- Research or experimental context
- Long-term deployment with learning needs

Production Considerations[1]

1. **Start Simple:** Begin with CoT, add complexity only when needed
2. **Measure Everything:** Track accuracy, latency, and cost per pattern
3. **Cache Aggressively:** Especially for CoT and tool responses
4. **Set Cost Limits:** Implement circuit breakers for expensive patterns
5. **A/B Test:** Compare patterns on real workloads
6. **Monitor Degradation:** Watch for pattern performance decay

Current Adoption: 51% of surveyed professionals are actively using agents in production, with 78% having active implementation plans[1].

Implementation Tips

- **Pattern Combination:** Many patterns work well together (e.g., CoT + Tools, Memory + Multi-Agent)
- **Fallback Strategy:** Implement graceful degradation from complex to simple patterns
- **Prompt Engineering:** Each pattern may require specific prompt formats
- **Evaluation Metrics:** Define clear success criteria before pattern selection
- **Framework Selection**[10-12]: Consider LangChain (63% adoption), AutoGen (Microsoft), or AutoGPT based on needs

Try It Yourself

````
### Get Started:
```bash
   pip install agentic-pattern
````

Quick Example:

```
from agentic_patterns import get_pattern, create_client

# First, create an LLM client
llm_client = create_client("openai", api_key="your-api-key")
# or
llm_client = create_client("google", api_key="your-gemini-key")

# Then get the pattern with the client
```
```

```
pattern = get_pattern("chain_of_thought", llm_client)
result = await pattern.execute("I have 1million in my acount, will I get a bank
```

```
=== ACTUAL OUTPUT ===
Response: Whether you'll get a $500,000 loan with $1,000,000 in your account dep

1. **Credit Score:**  Your credit history is paramount.  A high credit score (75

2. **Debt-to-Income Ratio (DTI):**  Lenders look at your existing debts (credit

3. **Loan Purpose:** The reason for the loan matters.  A loan for a well-establi

4. **Bank's Policies:** Different banks have different lending criteria. Some ma

5. **Collateral:** While you have $1,000,000 in your account, the bank might sti

6. **Income Verification:**  Even though you have a significant amount of money


**In short:** Having $1,000,000 in your account greatly increases your chances o

Cost: $0.0005
Pattern: ChainOfThoughtPattern
Success: True
```

Resources:

- 🚀 **GitHub Repository:** https://github.com/balrm/agentic-patterns
- 📚 **Full Documentation:** See README on GitHub
- 💡 **Banking Examples:** Check the `demo/` folder

Contribute: Add your own patterns or improvements via pull requests!

References

1. LangChain State of AI Agents Report 2024
2. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, NeurIPS 2022, arXiv:2201.11903
3. Kojima et al., “Large Language Models are Zero-Shot Reasoners”, 2022
4. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning”, arXiv:2203.11171
5. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”, NeurIPS 2023, arXiv:2305.10601
6. Besta et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models”, AAAI 2024, arXiv:2308.09687
7. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning”, NeurIPS 2023, arXiv:2303.11366
8. Shinn & Gopinath, “Reflecting on Reflexion”, March 2023
9. Liang et al., “Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate”, EMNLP 2024
10. LangChain Enterprise Adoption Survey 2024
11. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation”, Microsoft Research 2023
12. AutoGPT GitHub Repository Statistics, 2024
13. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools”, Meta AI 2023, arXiv:2302.04761
14. Berkeley Function Calling Leaderboard, 2024

15. Packer et al., “MemGPT: Towards LLMs as Operating Systems”, arXiv:2310.08560
16. Mem0 AI Agent Memory Benchmark Report, 2024
17. IBM AI Agent Memory Research, 2024
18. “MASTER: A Multi-Agent System with LLM Specialized MCTS”, arXiv:2501.14304
19. “Planning with MCTS: Enhancing Problem-Solving in Large Language Models”, OpenReview 2024
20. “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents”, arXiv:2505.22954
21. OpenAI, “Learning to Reason with LLMs”, 2024

Agentic Ai | Ai Agent Development | Ai In Finance | Automation | AI

Written by Balaram Panda

85 followers · 15 following

https://www.linkedin.com/in/balarampanda/

Follow

No responses yet

Write a response

What are your thoughts?

More from Balaram Panda

Balaram Panda

AI and Model Context Protocol in Banking: Transforming Financial...

The $1 trillion opportunity: How banks are revolutionizing operations with AI agents an...

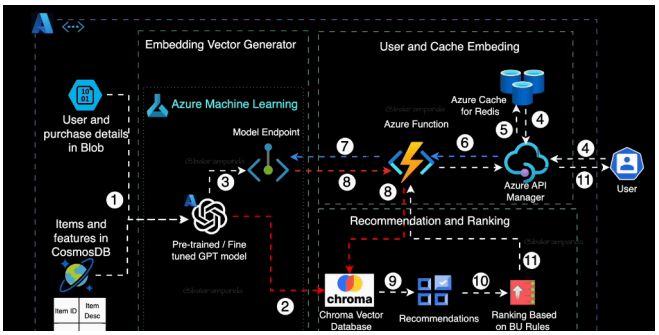
Jun 16 1

Balaram Panda

The Hidden Cost of AI Agents: Why ‘Free’ Isn’t Free

A CTO’s guide to the real costs of production AI agents in 2025

Aug 28 6



Balaram Panda

Generative AI based Recommendation Engine on Azur...

A high—level architecture for Generative AI based Recommendation Engine on Azure...

Sep 27, 2023 232 1

Balaram Panda

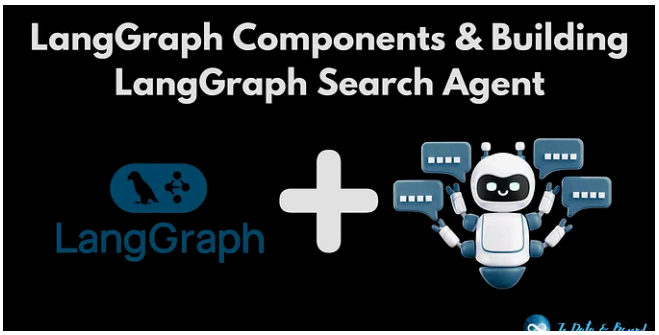
Top Vector Databases for Enterprise AI in 2025: Complete...

Best VDB for Revolution is Here

Jul 22

See all from Balaram Panda

Recommended from Medium

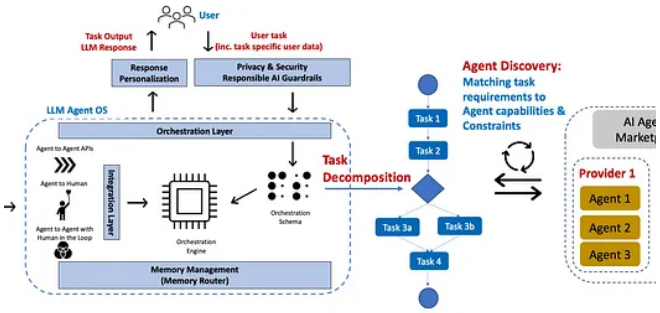


In Level Up Coding by Youssef Hosni

Building LLM Agents with LangGraph #3: LangGraph...

Step-by-Step Guide to Building LLM Agents with LangGraph

5d ago 183

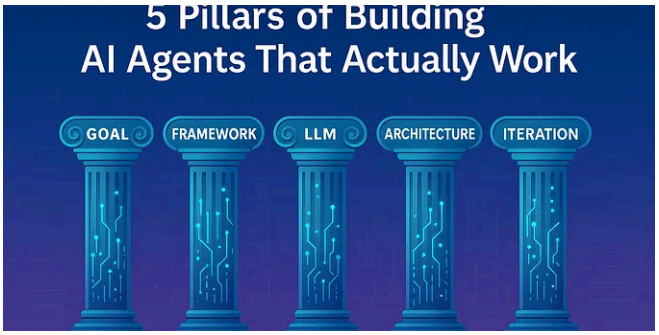


In Data Science Collective by Debmalya Biswas

Agentic AI MCP Tools Governance

Discovery and governance guidelines for AI Agents in the Enterprise

6d ago 175 4

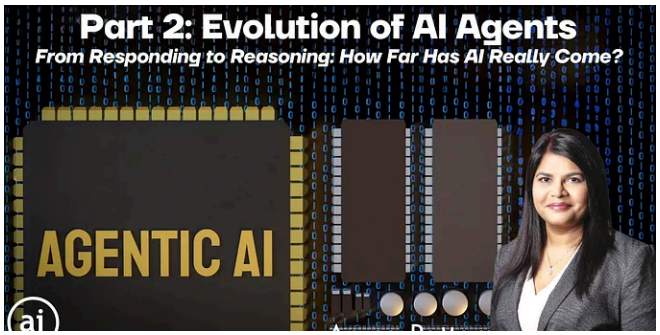


Lekha Priya

5 Pillars of Building AI Agents That Actually Work

There's a big difference between building an AI agent and building one that people actual...

Sep 2 6



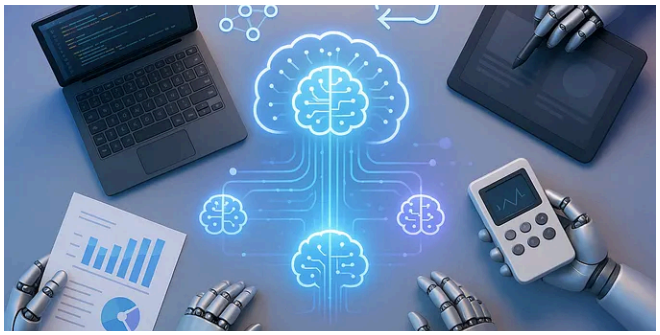
Aruna Pattam

Agentic AI: Part 2–Evolution of AI Agents

In Part 1, we explored what Agentic AI is, how it differs from traditional AI, and why it...

Jun 29 14

	Effort	Overhead	Complexity
Traditional RAG	Low	Low	Low
Contextual RAG	Medium	Medium	Medium
Context Engineering	High	Medium	Medium
Corrective RAG	Very High	High	High



In Towards AI by Vikram Bhat

Traditional RAG vs Context Engineering vs Corrective vs...

A practical framework for developers to choose the optimal RAG architecture based...

3d ago 6

Micheal Lanham

Multi-Agent AI Systems: The Complete Guide to Building...

Why one AI agent isn't enough for complex problems—and how to orchestrate intellige...

Jul 25 14 1

See more recommendations